PORTFOLIO

Vrishank Kuthiala

Eleven projects — internship work in backend and SEO engineering and e-commerce automation, plus a hackathon entry and family shop projects from my first semester at Vedam

Role · Python and Web Development

Organisation · NXtutors · CalmNCalculated · ThreadsWellThought · IntrikateInk · Vedam School of Technology

Period · June 2026 – October 2026

CONTENTS

1. About
2. The eleven projects
3. Technical skills
4. How I work
5. On AI assistance
6. Ajay Vatsyayan Classes
7. MathsBodhi
8. IIT JEE Home Tutors
9. Class Operations Agent
10. Blue Patch Initiative
11. ThreadsWellThought
12. CalmNCalculated — Planner Apps and Listing Images
13. AI-First Social Platform
14. AI-Assisted Development
15. QuantumRoute
16. IntrikateInk

1. About

I am a first-semester Computer Science student at Vedam School of Technology, and a member of the Vedam Tech Club and the Entrepreneurship Club. I have completed Vedam's **Tech KickStart – Build Your First Website** course (certificate issued 30 October 2026). I also attended the **"Managing AI at Work and Employees as Human-Machine Hybrids"** masterclass by StudyIn and Lancaster University (participation certificate, 28 October 2026). Since June 2026 I have also worked on real projects outside class: an internship, family e-commerce shops, and a hackathon entry.

The first was at **NXtutors**, an education company in Gurugram, where I worked on their websites and internal services from June to August 2026. I left to focus on my degree; the work is live and was handed over.

The second is **e-commerce** — building and running an online t-shirt business end to end, co-building planner apps with my dad and making their Etsy listing images, and connecting my mum's art shop to Etsy's API.

The third is academic: with five teammates I entered the **Smart India Hackathon 2026**, where I built the web platform for a route-optimization project.

What connects them is that almost none of this was practice work. These are live client websites, a real shop taking real orders, and a business's main source of customers. The habits that come from that — back everything up before touching it, rehearse every change, verify against the real system rather than the report about it — are what I would bring to a team.

Contact: vrishankkuthiala13@gmail.com · github.com/fierce-fly239 (personal) · github.com/vrishanknxt (work, NXtutors)

2. The eleven projects

Project	What it is	Status
Ajay Vatsyayan Classes	Recovered 4,732 pages Google refused to index on a live business website; built its deployment system	Live
MathsBodhi	Built search pages; resolved a twelve-hour outage; repaired a deployment system that had never reached visitors	Live
IIT JEE Home Tutors	Designed, built and launched a 1,860-page tutor discovery platform in Django, solo	**Live**, handed over
Class Operations Agent	A Python background service with 10 capabilities and 37 passing tests	Complete, handed over
Blue Patch Initiative	A website with a database-backed editing panel for a non-technical owner	Delivered
ThreadsWellThought	An automated pipeline taking a t-shirt from idea to a live, self-fulfilling online listing	Pipeline proven; shop pre-launch
CalmNCalculated	Planner apps co-built with my dad, and all of their Etsy listing images and videos	Live; BudgetOS hero still live
AI-First Social Platform	Architecture and costing for a sixteen-agent social platform, and the Phase 1 build	**Built, not launched**
AI-Assisted Development	The working system behind all of the above	In daily use
QuantumRoute	Smart India Hackathon 2026: a web platform comparing quantum-inspired and classical delivery-route planning	Submitted 23 Sep 2026
IntrikateInk	Connecting my mum's ten-year-old Etsy art shop to the Etsy API; social channel plan for the family's shops	Connected

3. Technical skills

I have split this deliberately, because a single list would overstate what I can do. I am starting my engineering degree, not finishing one.

What I can write myself — Python, at the level of reading and writing working code and following what it does. SQL and MySQL, enough to query a database, understand its structure and change it carefully. HTML, CSS and

Java, learnt at Vedam in my first semester (August to October 2026), and the basics of JavaScript (variables, conditionals, loops and arrays), which started on 30 October 2026 and which I have written in class. I have solved 67 LeetCode problems in Java (44 easy, 17 medium, 6 hard; as of 7 November 2026). Working at the command line on a Linux server.

What I have worked in, with AI writing the code — PHP and CodeIgniter, React and Node (JavaScript beyond the basics above), Django, Bash, and Kotlin for the Android version of my Owwsie (formerly Flow) finance app. I can read these well enough to know what a change does, decide whether it is the right change, and prove it worked afterwards. I could not sit down and write them from a blank file today. I expect to learn several of them properly at Vedam.

Systems I have worked on directly — AWS EC2, nginx, CloudPanel, pm2, GitHub Actions, SSH, Linux file permissions and process management. PostgreSQL, MySQL, SQLite, Supabase and Redis as the databases behind the projects in this portfolio. Etsy Open API v3, Printful API, Google Search Console, OAuth sign-in flows.

What I would actually claim as a skill — diagnosing a fault on a live system, backing up and proving the backup is real, rehearsing a change before making it, and verifying a result against the system itself rather than against a tool's report of it. These are the things in this portfolio that were mine, and they are the reason none of these sites broke.

4. How I work

Three habits, each of which came from something going wrong.

Back it up, then prove the backup. Before the largest set of changes on a client's main website, I took a complete copy — all 80,203 files, 173 MB of images that had never been copied off the server, and every database table. Then I checked the file counts matched, checked the row counts matched, and pulled one image back out of the archive and opened it to confirm it was real. A backup you have only listed and never opened is not a backup.

Rehearse every change. Every database change was run first in a mode that reports what it would do without doing it. Twice that rehearsal caught a change that would have published wrong information to a live business website — once sending an entire city's link to a single housing society, once trying to repair links from data that was itself already broken.

Say what you proved and what you guessed, separately. During a twelve-hour outage the team wanted to know whether the customer data had been lost. What I could actually prove was that the back end was not responding, which says nothing about the data. My message to the team said exactly that and no more, even though everyone wanted reassurance. The data turned out to be intact. The careful answer held up where a confident guess would have been remembered if it had been wrong.

5. On AI assistance

I use AI tools heavily, and this portfolio states that rather than hiding it. That does not mean I do not know what I am doing: I read and understand what the AI writes, and I decide whether it is the right change. One of the project files is specifically about the system I built around them.

The short version: an AI writes code faster than any junior developer, and it is also confidently wrong on a regular basis. Working with one productively is not about asking better questions — it is about building a structure that catches the mistakes. Mine is a knowledge base that survives between sessions, a rule that points at files rather than trusting recall, and verification against the real system rather than the tool's own report.

Several of the failures documented in these files are cases where a tool told me it had succeeded and reading the actual file proved it had not. Knowing not to trust an output you have not checked is the skill.

6. Ajay Vatsyayan Classes

The company's main source of customers had 4,732 pages that Google refused to list, and nobody knew why. I read the code rather than theorising, and found that three of the four problems came from a single function that built each page's web address.

The largest finding was not in Google's report at all: **two thirds of the website — 2,350 of 3,485 pages — carried a link to itself that led nowhere**, because the address was being built twice by two different rules that disagreed.

Fixed the stored data, then fixed the code so new pages cannot reintroduce it, then rewrote every page title from information already in the database. **All 16,545 internal links now resolve. Unique titles went from 1,181 to 3,485.** Also cut the homepage's immediate image loading from 198 images to 2, and built the automatic deployment system with backup and self-rollback that the owner had asked for.

7. MathsBodhi

Three problems, each hidden behind a confident but wrong signal.

The 27 search pages I built did not appear, and the team had concluded it was a database fault. It was not — the web server was serving one folder while the publishing system delivered into another.

Then the site went down for twelve hours while its own monitoring tool reported it healthy with eleven hours of uptime. The application had died one second after starting, inside a wrapper that stayed alive around it. The real error sat in a log file nobody had opened. **One setting was wrong. One command fixed it** — and no administrator rights were needed, contrary to my own first conclusion.

Then publishing the founder's section revealed that **the deployment system had never once delivered to visitors.** The public front end had been three weeks out of date. Two separate faults, both present since it was written, both now fixed.

8. IIT JEE Home Tutors

A nationwide tutor discovery platform built alone in two days: **1,860 published pages**, 126 locations arranged as a tree, all seven types of machine-readable data the client specified, and a working administration panel for his team.

The central design problem was content. Publishing 1,860 nearly identical pages is the fastest way to be ignored by Google — I had watched exactly that happen to 1,940 pages on the company's other website. The answer was **reusable content blocks**, written once per subject-and-exam combination and appearing on every page they apply to.

I also moved the live-database switch off the deployment checklist and built it early, testing both paths — because a checklist item is a promise that someone remembers it under pressure, and the company's other site had gone down that same day from exactly that class of problem.

The site went live on 31 July, deployed end to end by me: server, domain, database, security certificate. The moment I would point to is a check that nearly did not happen. My own deployment guide said all 1,860 pages rebuild automatically from the setup scripts. I tested that claim against an empty database instead of trusting it, and got **one page**. Everything else rebuilt; the pages did not. Following my own document would have launched a one-page site on a domain the client had bought four days earlier. A document written by the person who built the

thing is not evidence.

In August the client read the live site and said the pages were too thin. He was right, so I measured it: 625 words a page, 45 of them unique. Three days and two deployments later the structure was rebuilt around city and subject hubs with every page linking to its neighbours, and **62 blocks of subject-and-exam-specific writing took pages to 2,691 words** — all 1,860 clearing his target. That work also uncovered that **every "Book a demo" button on the site had been dead since launch**, 5,580 of them, so nobody could actually make an enquiry.

With that model in place, taking the platform national was a data job rather than a writing job: **340 cities across 27 states, imported in nine seconds**, each inheriting all sixteen content sections automatically. The site went from 1,874 addresses to **7,298**. What a plain import would have got wrong is the part I would defend — Gurgaon was already on the site under its other name and would have been duplicated, and every page outside Gurgaon had to stop promising a tutor who could come to the house. I checked that rule across all 7,298 addresses individually rather than sampling, because one page promising a home visit in a city with no tutors fails exactly the parent it attracted.

The part worth reporting is what I told him he had not asked to hear. He also wanted pages 90% distinct from one another; I measured that, and the extra content made the *percentage* worse, because shared writing dilutes the ratio even when nothing gets less unique. Depth is something code can produce. Distinctness needs real facts about real places, and those have to come from him — which is a far smaller ask than the five and a half million words his brief implied.

That warning came true, and I have to report it as such. On 18 August the client asked whether the pages were indexed. **One is** — the homepage — out of 7,298. No technical fault: the robots file, the sitemap, the server speed and the links all check out, and Google has the addresses. It is simply not fetching them. Measuring the pages against each other told me why it might not bother: **any two city pages on the same subject are 94% identical.** The reusable-content model gave every page depth, which is what it was for, and made 7,140 of them near-copies in the process.

The honest caveat, which I only reached because I questioned my own first answer: the site is **eighteen days old**. At eighteen days this is close to normal, and age by itself explains everything I saw. So I gave the client the numbers and said it is still early, instead of a confident diagnosis. It gets re-checked in September, and that re-check is what actually settles it.

9. Class Operations Agent

A Python background service merging two planned services into one, built to match an existing production pattern: **10 capabilities, 37 passing tests, a 22-case stress suite**, controlled database migrations, personal-information masking on the AI path, and three levels of access control.

Its best design decision was moving the AI connection inside the error handling, so that **8 of the 10 capabilities work fully with no AI key at all**. The company's other service uses a different AI provider, so the assumed key was never going to be available.

10. Blue Patch Initiative

A single-page website whose editing panel let the owner make changes that only ever saved to his own computer — nobody else would have seen any of them. I connected it to a hosted database so edits publish properly, then built an escape route for the content conflict that creates, before it happens.

Six requested changes delivered from a spoken brief in a mixed language, all verified. Along the way I found a fault nobody had noticed: **if one script failed to load, the entire page stayed blank.**

11. ThreadsWellThought

A print-on-demand t-shirt business built from nothing, with a pipeline behind it: a design idea becomes a live listing with **20 variations, 6 images and 13 search tags**, and every order after that is printed and posted automatically.

The obvious approach was closed off — the printing company's interface cannot create listings for a shop connected to this marketplace — so the design had to be inverted. Proven end to end on a real published listing, not a test. **Built with zero new spending.**

The shop has not opened yet, and that is deliberate. The chain was proven on a test listing, which came down once it had done its job; the first real listing waits on the artist. Two genuine obstacles are still open though — an outstanding Indian tax registration and a developer setting that hides listings from search — and I only found the second by opening the public page a customer would see, after weeks of reading "0 views" on a dashboard.

Two lessons I kept. The marketplace accepted wrongly-formatted data, kept only part of it, and still reported success — nothing failed and nothing warned, so **after every write to someone else's system, read it back and check what actually landed**. And **zero is a different kind of number from one**: it usually means something is switched off, not unpopular.

12. CalmNCalculated — Planner Apps and Listing Images

CalmNCalculated is the family's Etsy shop for digital planner applications — single-file dashboards that open in a browser and work offline. The family has 16 of them. I **co-built all of the planner apps with my dad, Amit**, with AI writing much of the code, and I **made all of the listing images and videos myself**. I also created the listings by hand before the Etsy API pipeline existed, and the shop account is on my personal email.

For BudgetOS, where the images are the entire sales pitch because there is nothing to photograph, I made **10 images and 2 videos** in ChatGPT and Veo, structured as a sequence rather than a gallery: an opening image, a seven-card feature tour, a device comparison, and two videos. For the other planners I made the hero images in two levels — a scene, then banner and badge edits on top — and **11 heroes were swapped onto live listings on 14 August 2026**.

Since 6 September 2026 heroes are built from a code template instead, which was my decision, made after I brought a competitor's listing as a reference.

The BudgetOS listing was later refreshed onto a shop-wide template, which replaced most of its set. **My BudgetOS hero image is still live and still first.** I also kept no record of how the original set was made, and that cost someone else time later; it is written up in the project file.

13. AI-First Social Platform

The client asked for a plan for a social network where every user is helped by a team of AI agents — sixteen named in his brief, with a stated target of fifty to a hundred.

Two findings shaped the plan. **Seven of the sixteen agents did not need AI at all** — feed ranking, recommendations, trend detection and topic grouping are statistics and ranking problems, older and faster and better understood than asking a language model, and routing them through one would have cost several times more

for worse results. And having costed every agent individually, **the platform as briefed would cost roughly six times more to run than an app of its size earns**. I put the arithmetic in the plan along with three changes that bring it to about a quarter.

The central recommendation is deliberately not what was asked for: build three agents, not a hundred, and find out whether people use it before building the rest. Ten decisions in the plan contradict or extend the client's brief; all ten are listed in an appendix with the reasoning, rather than quietly implemented.

The client came back asking to narrow it to a **pilot in one city**, open it to **anyone** rather than a student niche, and cut the cost. Rewriting it for those turned up a mistake of my own: my cost figure had been calculated using three agents that my own phasing table put in a later phase and which do not run in the pilot. Costing only what actually ships took the pilot from about Rs. 16,000 a month to **about Rs. 3,700**. I put the correction in the document's appendix rather than quietly swapping the number, since he had already read the wrong one.

The plan was approved on 18 August, at a 1,000-user pilot — the same number the costing already assumed, so nothing had to be re-worked to accept it — **and phase one was built the same day**: Django and PostgreSQL, 36 automated tests, verified screen by screen in a browser.

The part I would point to is that the "costs nothing until it is public" promise is enforced by the code rather than only written in the plan. The AI agents sit behind a swappable backend whose default makes no network calls and needs no key, so the whole application and every test run for free; one line in a config file switches to the paid service. The safety check works the same way — ordinary pattern matching settles about 95% of posts for nothing, and only genuinely ambiguous text costs an AI call.

Testing that safety check found a hole worth describing. I fold the digit 1 to the letter i so a slur written as "mus1ims" is still caught; my test failed, because 1 also stands in for l. So I checked both readings — and it still failed, on "these mus1ims should be k1lled", which needs 1 read as *l* to see the community word and as *i* to see the threat. Neither reading on its own contains both halves. The fix pools the matches across readings. The first fix had looked complete and passed its own test; the gap was in the shape of the check, not its coverage.

I then took it from "it runs" to "it could be public": the production security settings, rate limits, and the terms, privacy and complaints pages that Indian law requires of a platform hosting other people's content. That last one includes a gate I would defend anywhere — the application **refuses to start in production until a real grievance officer is named**, and I left the fields empty rather than inventing a name to make my own check pass. It is the client's decision, and an invented compliance detail is worse than a missing one.

The launch content follows the same line. There are fifty real arguments ready for day one, and not one fabricated reply or vote — seeding a question is a prompt, seeding replies from people who do not exist is astroturfing, and it would have corrupted the pilot's own measurement, since what the pilot measures is replies.

14. AI-Assisted Development

The system behind everything above: a knowledge base holding **324 registered source files, 11 compiled reference articles, 27 dated changelog entries and 5 live codebases**, so that no session starts by re-explaining what happened last time.

Its most useful rule came from a failure. A task list kept losing items between sessions, because it was being rebuilt from whatever was recent rather than read from the file that held it. The fix was not to try harder — it was to name one file as the only source of truth and require it to be read. **Point at a file, do not rely on recall.** That principle now runs through all of this work.

15. QuantumRoute

Smart India Hackathon 2026, problem SIH26137, "Quantum-Inspired Intelligent Traffic Route Optimization", set by Egreen Quanta. I was one of six on team "Cyber pookies_26"; our team leader Anant Saboo made the submission from his login. **I built the web platform** — a Python backend and a React frontend — with AI help.

It plans delivery-van routes on Gurugram road networks of 9 and 49 stops, comparing QPSO (a quantum-behaved particle swarm method from 2004; it runs on ordinary computers, with no quantum hardware) against classical PSO. Over 30 runs, QPSO's routes came within **0.60%** of the proven best on the small network against 1.20% for PSO, and QPSO won **20 of 30** runs on the larger one. There are 94 automated tests.

The finding I would keep: **QPSO's published settings cost about 12%** on this problem, and we tuned them — then checked our own comparison and found we had accidentally given QPSO 40 more evaluations than PSO. We fixed it, and both corrections moved the result against us. Also, a build prompt I was sent mixed up "quantum rotation gates", a different family of algorithms, with QPSO; we kept them separate.

Presented at Vedam's internal round on 10 September 2026 and submitted on 23 September (Idea ID 150607, Team ID 159870). Code: github.com/fierce-fly239/quantumroute (MIT).

16. IntrikateInk

IntrikateInk is my mum Surabhi's ten-year-old Etsy art shop: 763 sales and 83 reviews at 5.0. The revival plan is my dad's. My part was the Etsy connection and the social channel plan.

On 17 July 2026 I decided to use Etsy's API directly instead of a Make.com pipeline. When we ran the sign-in with my mum it failed with an `invalid_target` error. It turned out to be a fault on Etsy's side that other developers were reporting too, so we waited rather than keep changing settings; once Etsy fixed it we connected, the shop's ID resolved and a read test passed.

I set the audit of the shop's 28 live listings to **report-only**, so nothing is written to a shop with that many five-star reviews. On 12 August 2026 I set the social channel plan for all three of the family's shops: Buffer for Pinterest, Instagram and TikTok, with Facebook covered by crossposting from Instagram. That crossposting is not yet proven for posts sent through Buffer. Pinterest is now live.